Proyecto #1

ALGORITMOS DE REGRESIÓN

- Esteban Martínez
- Javier Osorio
- Cesar Sosa

Universidad Modelo
Escuela de Ingeniería
Ingeniería en Desarrollo de Tecnología y Software

Asignatura: Inteligencia Artificial
Profesor: M.C. Edson G. Estrada López
28 de septiembre de 2026

ÍNDICE

- Objetivos
- Descripción general del proyecto
- Requerimientos solicitados vs. implementados
- Marco teórico
  - Gradient Descent: Stochastic y Mini-Batch
  - Escalamiento de Features: normalización y estandarización
- Implementación
- Pruebas y demostración
- Bugs conocidos
- Conclusiones
- Comentarios de los integrantes
- Referencias

OBJETIVOS

Objetivo general:
Implementar desde cero, en Python, las variantes Stochastic y Mini-Batch de Gradient Descent y las técnicas de escalamiento de features (normalización y estandarización), que se usan para entrenar modelos de regresión.

Objetivos particulares:

1. Implementar una regresión lineal con Stochastic y Mini-Batch Gradient Descent y comparar los parámetros obtenidos con Batch Gradient Descent y la Ecuación Normal.
2. Programar la normalización y la estandarización sin librerías y verificar que den los mismos resultados que Scikit-Learn.

DESCRIPCIÓN GENERAL DEL PROYECTO

El proyecto consiste en implementar desde cero, en Python, dos variantes de Gradient Descent y dos técnicas de escalamiento de features, y comparar los resultados con métodos de referencia. Se trabajó en tres notebooks de Jupyter:

Stochastic Gradient Descent

Ajusta una regresión lineal a 200 datos sintéticos actualizando θ con una sola muestra a la vez. El resultado se compara con Batch Gradient Descent y la Ecuación Normal.

Mini-Batch Gradient Descent

Ajusta la misma regresión usando mini-lotes de 20 muestras y compara el resultado con Batch Gradient Descent y la solución exacta.

Feature Scaling

Normaliza y estandariza un dataset de 10 personas (edad, salario y horas) con funciones propias y verifica el resultado con Scikit-Learn.

Herramientas: Python, Jupyter Notebook, NumPy, pandas, Matplotlib y Scikit-Learn.

REQUERIMIENTOS SOLICITADOS VS. IMPLEMENTADOS

Tabla 1. Requerimientos solicitados vs. implementados.

Requerimiento	Notebook	Estado
Generar un dataset aleatorio para regresión lineal	SGD / Mini-Batch	Implementado
Implementar Stochastic Gradient Descent desde cero con learning schedule	SGD	Implementado
Implementar Mini-Batch Gradient Descent desde cero	Mini-Batch	Implementado
Comparar contra Batch Gradient Descent y la Ecuación Normal	SGD / Mini-Batch	Implementado
Graficar el costo por iteración y la recta de predicción	SGD / Mini-Batch	Implementado
Generar un dataset con features de escalas distintas	Feature Scaling	Implementado
Implementar la normalización (Min-Max) sin librerías	Feature Scaling	Implementado
Implementar la estandarización (Z-score) sin librerías	Feature Scaling	Implementado
Evitar la división entre cero cuando el rango o σ es 0	Feature Scaling	Implementado
Verificar los resultados contra Scikit-Learn	Feature Scaling	Implementado

GRADIENT DESCENT

¿Qué es?

$$\theta_{t+1} = \theta_t - \eta \nabla \theta L(\theta_t)$$

- Es un algoritmo de optimización que busca los valores de los parámetros que minimizan la función de pérdida de un modelo.
- En cada paso calcula el gradiente de la pérdida y mueve los parámetros en la dirección contraria, es decir, "cuesta abajo".
- La versión clásica, Batch Gradient Descent, usa todo el conjunto de datos para calcular cada actualización. Esto da pasos estables, pero es lento cuando hay muchos datos.
- Para resolver esto existen variantes que usan solo una parte de los datos en cada paso: Stochastic Gradient Descent y Mini-Batch Gradient Descent.

STOCHASTIC GRADIENT DESCENT

¿Qué es?

- Es un algoritmo de optimización que actualiza los parámetros de un modelo usando una sola muestra de datos a la vez en lugar de todo el conjunto.

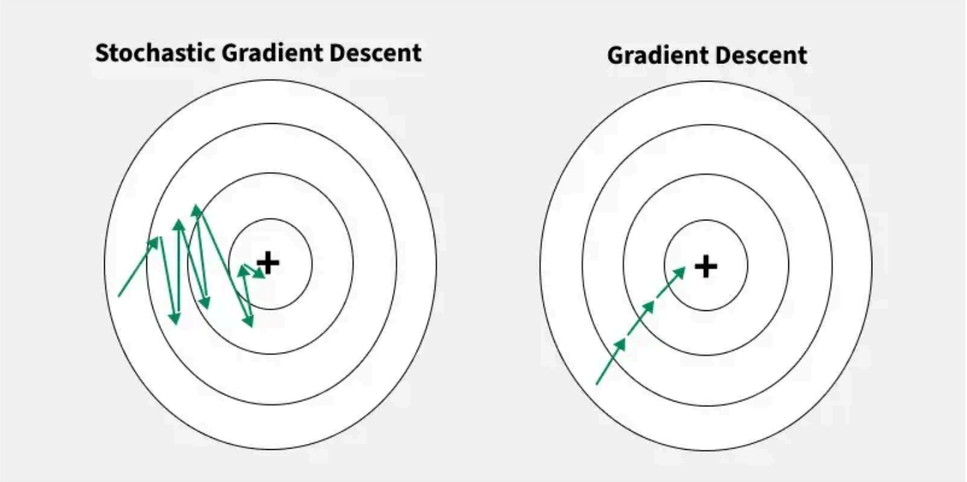


Figura 1. Trayectoria de Stochastic Gradient Descent vs. Gradient Descent.

STOCHASTIC GRADIENT DESCENT

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t; x_i, y_i)$$

- θ (theta): los parámetros del modelo (pesos y sesgos) que quieres aprender.
- t: el paso o iteración actual. θ_t son los parámetros ahora, θ_{t+1} los parámetros después de actualizar.
- η (eta): la tasa de aprendizaje (learning rate). Controla qué tan grande es cada paso.
- L: la función de pérdida (loss), que mide qué tan mal predice el modelo.
- ∇θ L: el gradiente, es decir, el vector de derivadas parciales de la pérdida respecto a cada parámetro.
- (x_i, y_i): un solo ejemplo de entrenamiento elegido al azar.

STOCHASTIC GRADIENT DESCENT

Ventajas

- Es más fácil de ajustar en la memoria, ya que la red procesa una sola muestra de entrenamiento a la vez.
- Es computacionalmente rápido, porque solo se procesa una muestra a la vez.
- En conjuntos de datos más grandes puede converger más rápido, ya que provoca actualizaciones más frecuentes de los parámetros.
- Debido a las actualizaciones frecuentes, los pasos que se dan hacia el mínimo de la función de pérdida presentan oscilaciones, que pueden ayudar a salir de mínimos locales de la función de pérdida (en caso de que la posición calculada resulte ser un mínimo local).

STOCHASTIC GRADIENT DESCENT

Desventajas

- Debido a las actualizaciones frecuentes, los pasos que se dan hacia el mínimo son muy ruidosos. Esto a menudo puede desviar el descenso de gradiente hacia otras direcciones.
- Además, por los pasos ruidosos, puede tardar más en lograr la convergencia hacia el mínimo de la función de pérdida.
- Las actualizaciones frecuentes son costosas computacionalmente, ya que se usan todos los recursos para procesar una sola muestra de entrenamiento a la vez.
- Pierde la ventaja de las operaciones vectorizadas, ya que trabaja con un solo ejemplo a la vez.

STOCHASTIC GRADIENT DESCENT

¿Cuándo utilizarlo?

1. Datasets muy grandes.
2. Aprendizaje en línea (online learning).
3. Entrenamiento de redes neuronales profundas.
4. Funciones de costo no convexas.
5. Recursos de cómputo limitados.
6. Modelos lineales a gran escala.

MINI-BATCH GRADIENT DESCENT

¿Qué es?

- Es un algoritmo de optimización que divide el conjunto de datos en grupos pequeños (mini-lotes) y actualiza los parámetros del modelo después de procesar cada grupo.
- Es un punto medio entre Batch Gradient Descent, que usa todos los datos, y Stochastic Gradient Descent, que usa una sola muestra.
- Los tamaños de mini-lote más comunes son potencias de 2, como 32, 64 o 128.

MINI-BATCH GRADIENT DESCENT

$$\theta = \theta - \alpha \frac{1}{b} \sum_{k=t}^{t+b-1} \nabla_{\theta} L(f(x^{(k)}; \theta), y^{(k)})$$

- θ: Los parámetros o pesos del modelo que se van a actualizar.
- α: La tasa de aprendizaje (learning rate), que define el tamaño del paso.
- b: El tamaño del mini-lote (batch size), es decir, el número de muestras que se procesan juntas.
- ∑: La suma de los gradientes calculados para cada una de las muestras k dentro del mini-lote actual.
- $\nabla \boldsymbol{\theta L}$: El gradiente de la función de pérdida (loss function) con respecto a los parámetros θ para una muestra individual $x(k)$, $y(k)$.

MINI-BATCH GRADIENT DESCENT

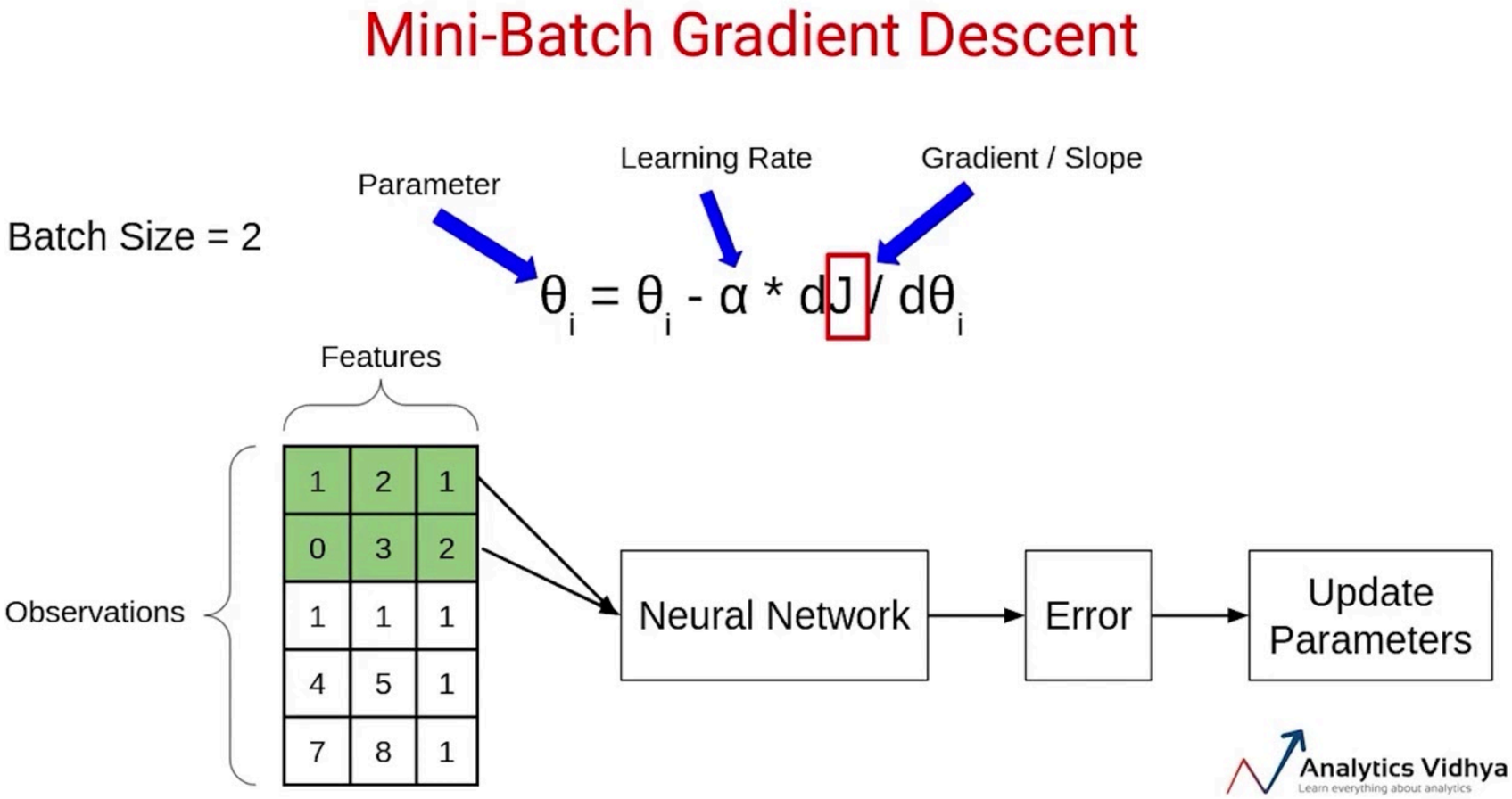


Figura 2. Diagrama del funcionamiento de Mini-Batch Gradient Descent. Fuente: Analytics Vidhya.

MINI-BATCH GRADIENT DESCENT

Ventajas:

1. Las actualizaciones son menos ruidosas que en SGD, por lo que la convergencia es más estable.
2. Aprovecha las operaciones vectorizadas y el hardware como las GPU, ya que procesa varias muestras al mismo tiempo.
3. No necesita cargar todo el conjunto de datos en memoria.

   Desventajas:
4. Agrega un hiperparámetro más, el tamaño del mini-lote (b), que hay que ajustar.
5. Todavía presenta algo de ruido, por lo que puede oscilar cerca del mínimo si la tasa de aprendizaje no se reduce.

COMPARACIÓN DE LAS VARIANTES

Tabla 2. Comparación de las variantes de Gradient Descent.

Característica	Batch GD	Stochastic GD	Mini-Batch GD
Muestras por actualización	Todo el conjunto (n)	1	b (ej. 32, 64, 128)
Velocidad por actualización	Lenta	Muy rápida	Rápida
Estabilidad de la convergencia	Muy estable	Ruidosa	Moderada
Uso de memoria	Alto	Bajo	Medio
Aprovecha vectorización	Sí	No	Sí
Uso recomendado	Datasets pequeños	Datasets muy grandes y online learning	Redes neuronales y la mayoría de los casos

ESCALAMIENTO DE FEATURES

Ajustar las variables numéricas para que queden en escalas comparables.

Ejemplo: la edad va de 18 a 65 y el salario de 14,000 a 34,000.

Sin escalar, la variable más grande "domina" solo por su tamaño, no por su importancia.

NORMALIZACIÓN

Reescala cada feature para que sus valores queden en un rango fijo, normalmente [0, 1].

Ventajas:

1. Garantiza un rango acotado y conocido, lo cual es útil en redes neuronales y en procesamiento de imágenes (píxeles de 0 a 255).
2. Conserva la forma de la distribución original.

   Desventajas:

3. Es muy sensible a los valores atípicos (outliers): un solo valor extremo comprime al resto de los datos en un rango pequeño.

NORMALIZACIÓN

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

x' valor normalizado (entre 0 y 1)

x valor original

x_{min} valor mínimo de la columna

x_{max} valor máximo de la columna

NORMALIZACIÓN

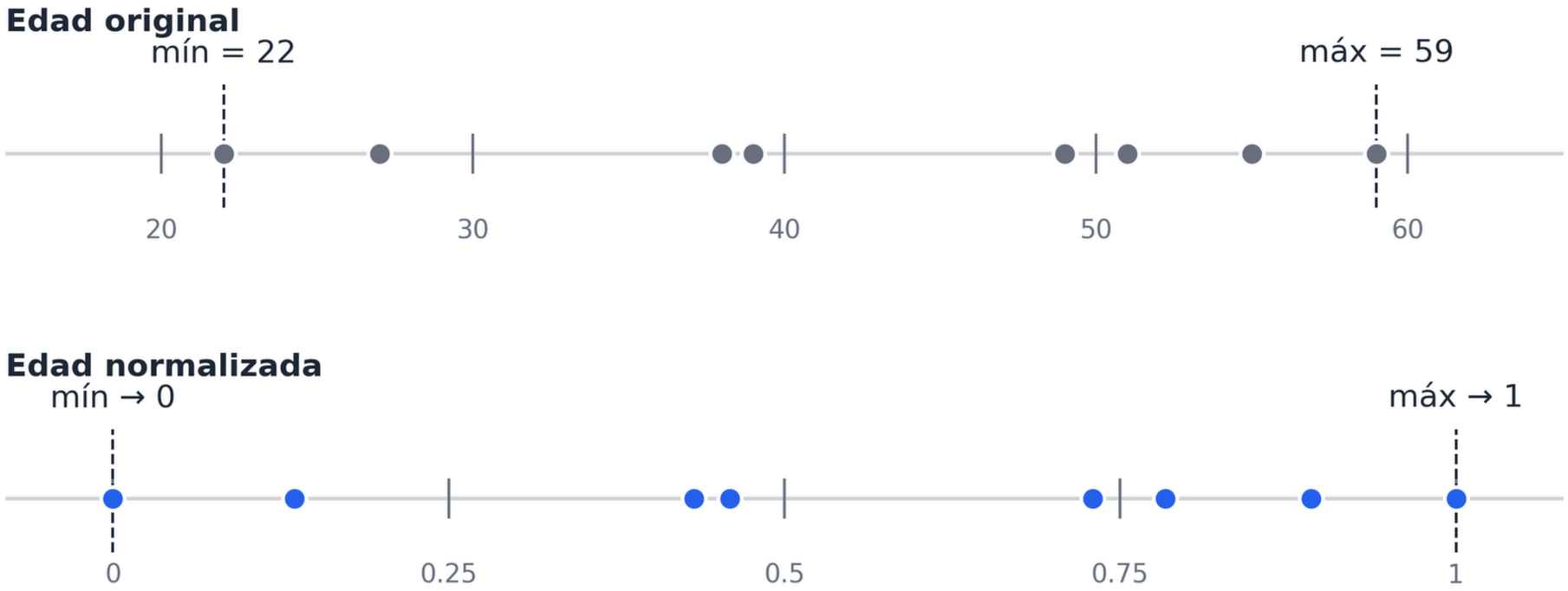


Figura 3. Edad original y edad normalizada con Min-Max.

ESTANDARIZACIÓN

Transforma cada feature para que tenga media 0 y desviación estándar 1.

Ventajas:

1. Es menos sensible a outliers que la normalización.
2. Es la opción preferida para algoritmos que suponen datos centrados o con distribución aproximadamente normal, como SVM, regresión logística y PCA.

   Desventajas:

3. El resultado no queda acotado a un rango fijo.

ESTANDARIZACIÓN

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

n	número de datos (10 personas)
x_i	cada valor de la columna
$\sum$	suma de todos los valores
μ	media: el centro de los datos
$x_i - \mu$	distancia de cada valor a la media
σ	desviación estándar: qué tan dispersos están

ESTANDARIZACIÓN

$$z = \frac{x - \mu}{\sigma}$$

- x valor original
- μ media (promedio) de la columna
- σ desviación estándar de la columna
- z valor estandarizado (cuántas σ se aleja de la media)

ESTANDARIZACIÓN

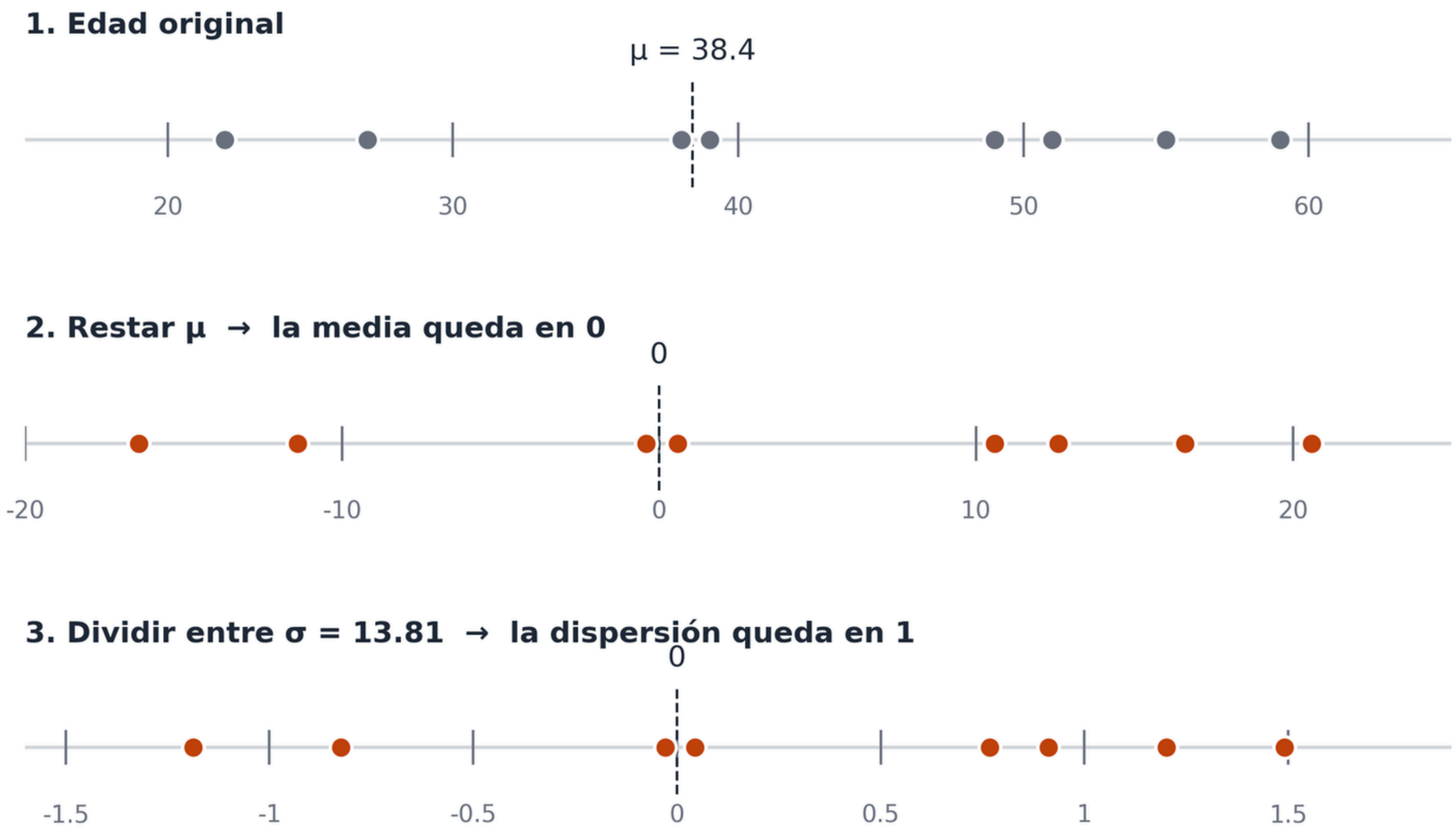


Figura 4. Proceso de estandarización de la edad (μ = 38.4, σ = 13.81).

IMPLEMENTACIÓN: STOCHASTIC GRADIENT DESCENT

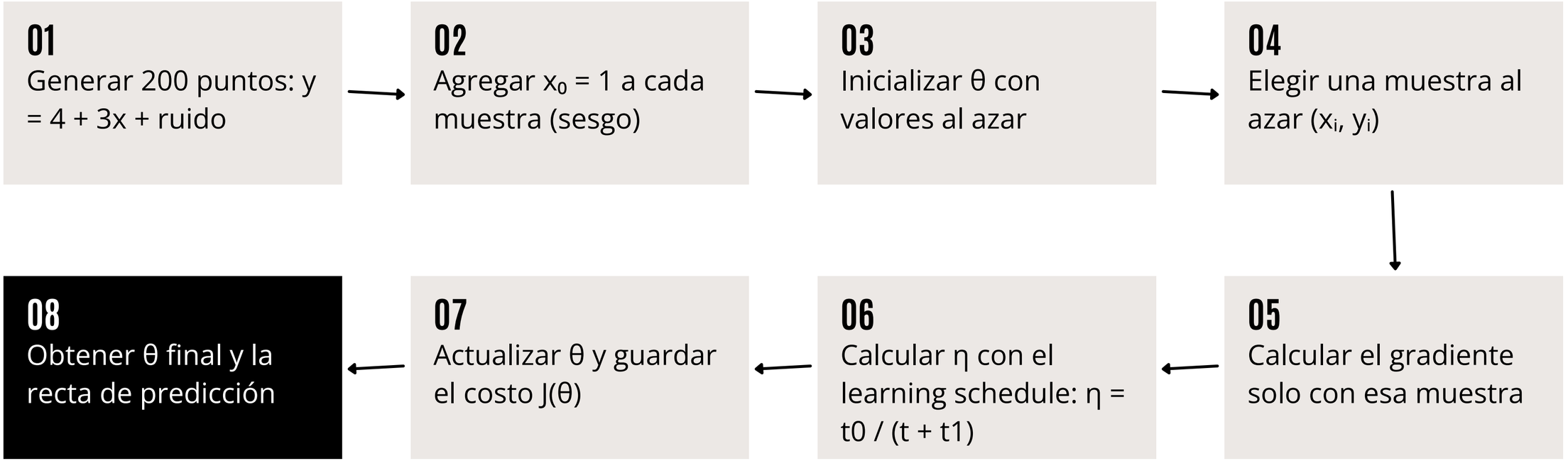


Figura 5. Diagrama de flujo de la implementación de Stochastic Gradient Descent.

Los pasos 4 a 7 se repiten 50 épocas × 200 muestras = 10,000 iteraciones. Hiperparámetros: t0 = 5, t1 = 50.

IMPLEMENTACIÓN: MINI-BATCH GRADIENT DESCENT

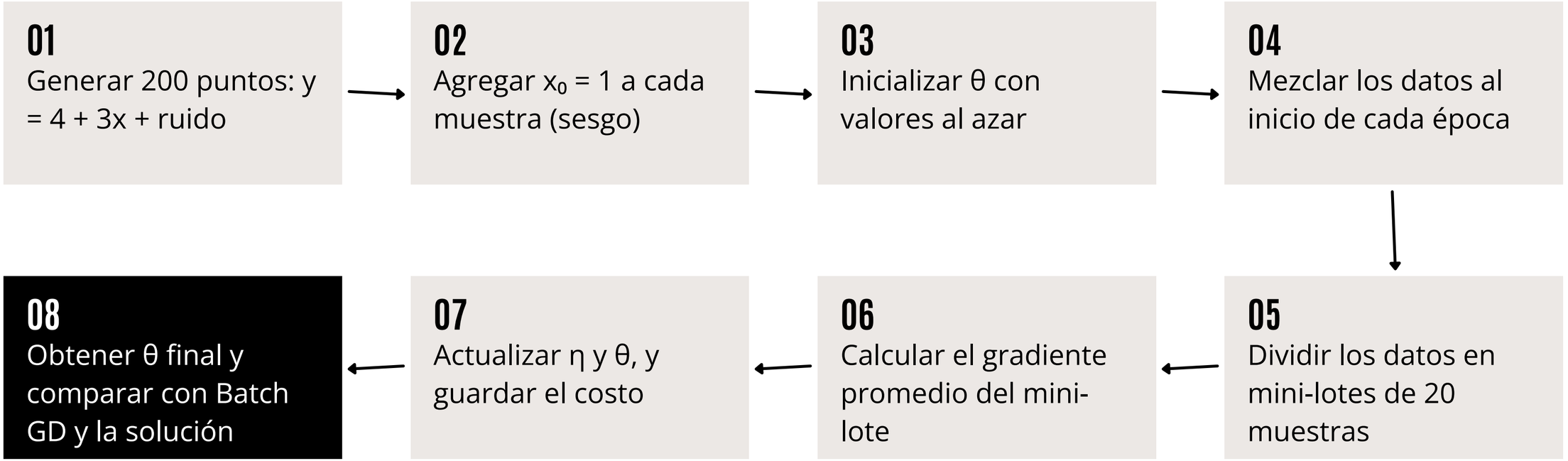


Figura 6. Diagrama de flujo de la implementación de Mini-Batch Gradient Descent.

Los pasos 4 a 7 se repiten 1,000 épocas × 10 mini-lotes = 10,000 actualizaciones. Hiperparámetros: b = 20, t0 = 5, t1 = 50.

IMPLEMENTACIÓN: FEATURE SCALING

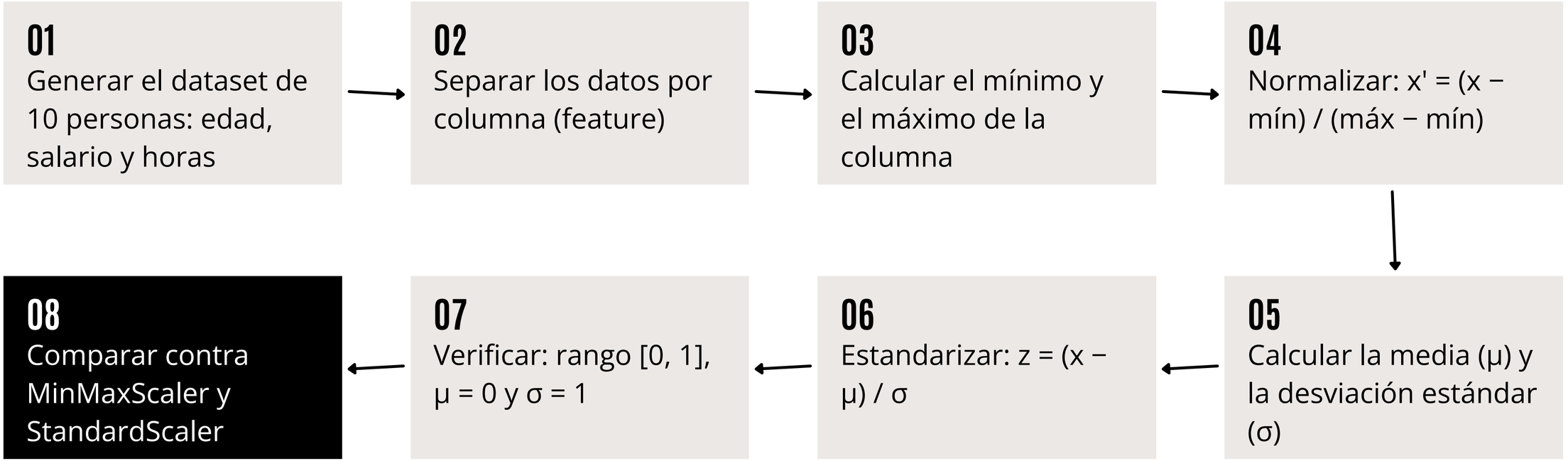


Figura 7. Diagrama de flujo de la implementación de la normalización y la estandarización.

Los pasos 3 a 6 se aplican a cada una de las 3 columnas. Si el rango o σ es 0, se asigna 0 para no dividir entre cero.

PRUEBAS: STOCHASTIC GRADIENT DESCENT

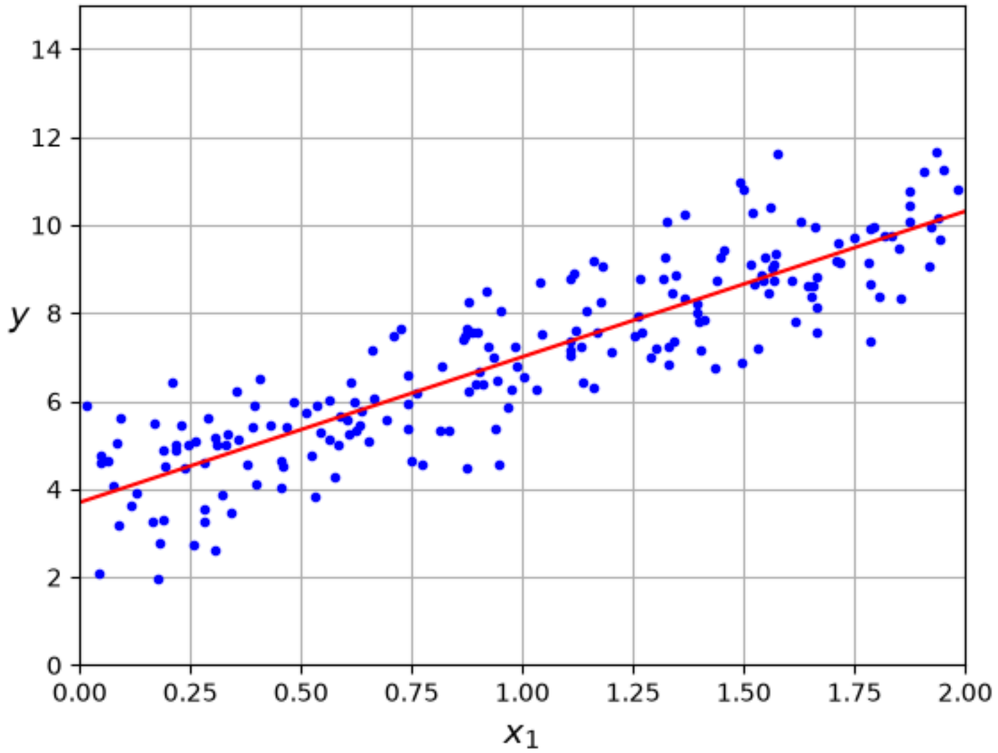


Figura 8. Recta obtenida con SGD sobre los 200 datos.

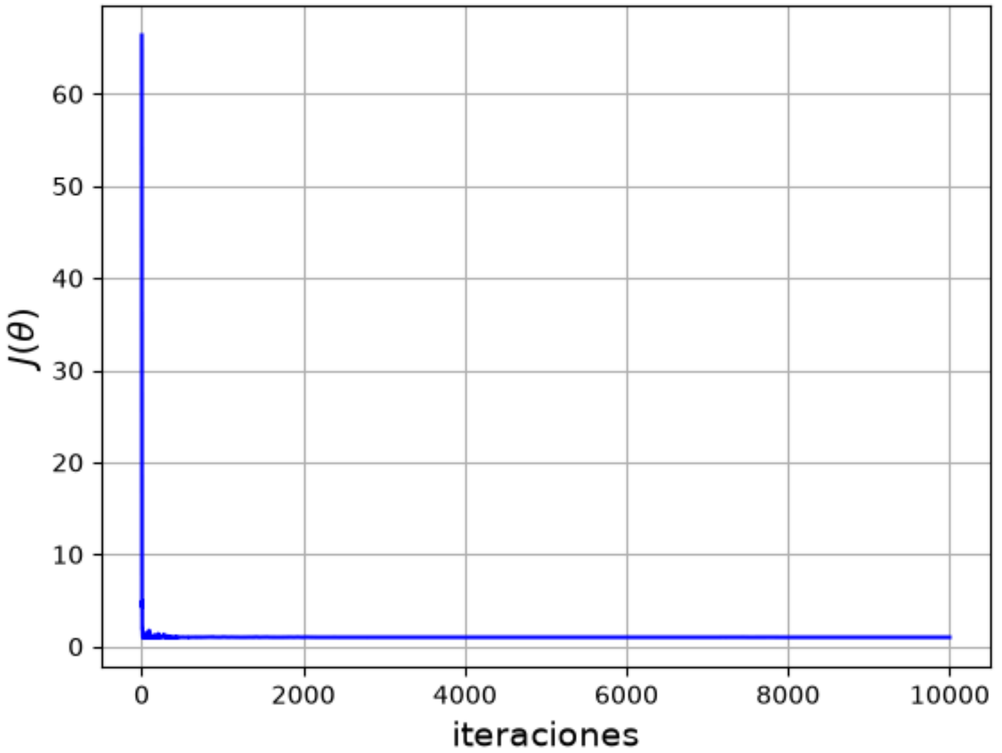


Figura 9. Costo J(θ) durante las 10,000 iteraciones de SGD.

Resultado: $\theta_0 = 3.698$ y $\theta_1 = 3.307$. La recta sigue la tendencia de los datos; el costo baja muy rápido en las primeras iteraciones y después se estabiliza.

PRUEBAS: MINI-BATCH GRADIENT DESCENT

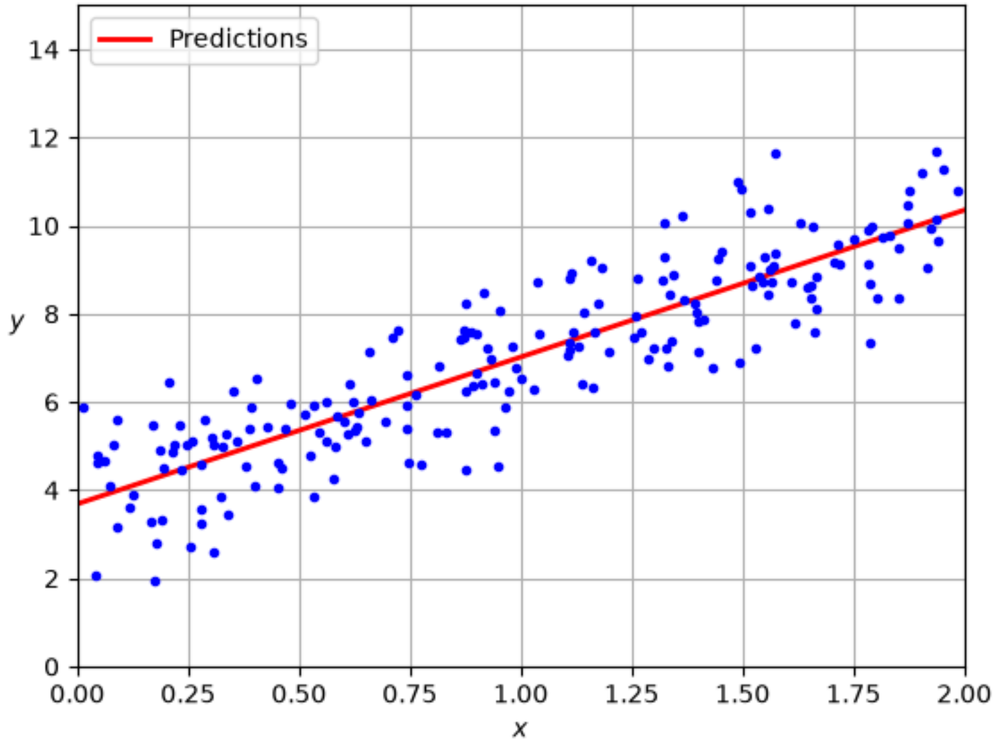


Figura 10. Recta obtenida con Mini-Batch.

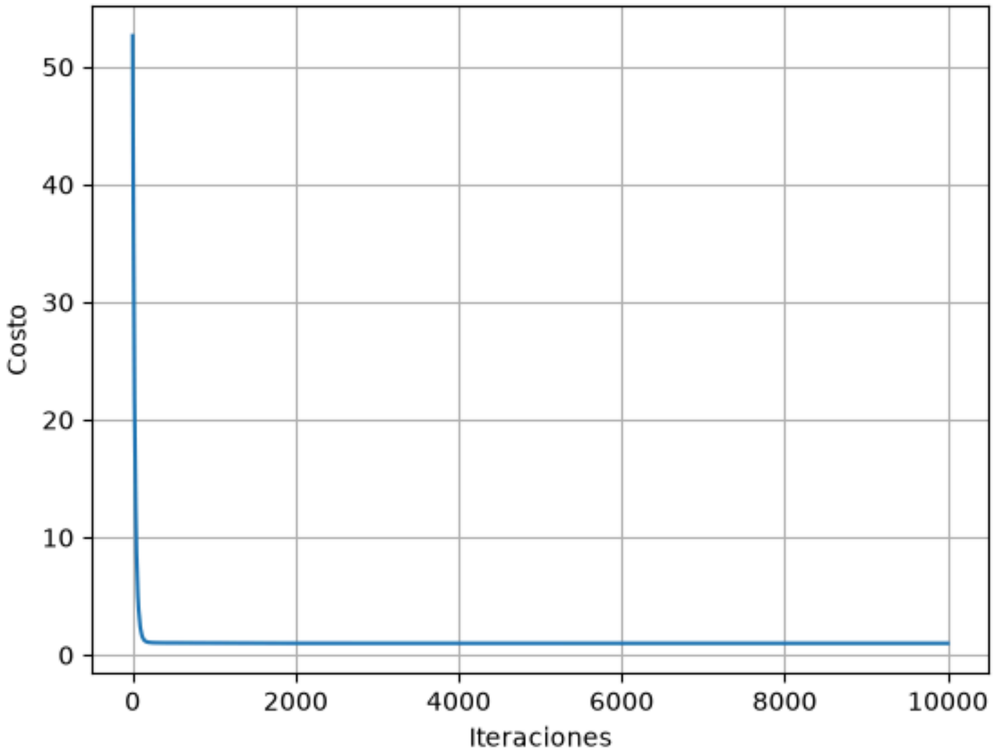


Figura 11. Costo en las 10,000 actualizaciones.

Resultado: $\theta_0 = 3.686$ y $\theta_1 = 3.334$, muy cerca de la solución exacta. Con mini-lotes de 20 muestras, el costo baja de forma estable, casi igual que en Batch Gradient Descent.

PRUEBAS: COMPARACIÓN DE RESULTADOS

Tabla 3. Parámetros obtenidos por cada método.

Método	θ_0	θ_1	Actualizaciones
Parámetros reales	4	3	—
Ecuación Normal (exacta)	3.6908	3.3296	—
Batch Gradient Descent	3.6908	3.3296	1,000
Stochastic Gradient Descent	3.6983	3.3075	10,000
Mini-Batch Gradient Descent	3.6863	3.3335	10,000

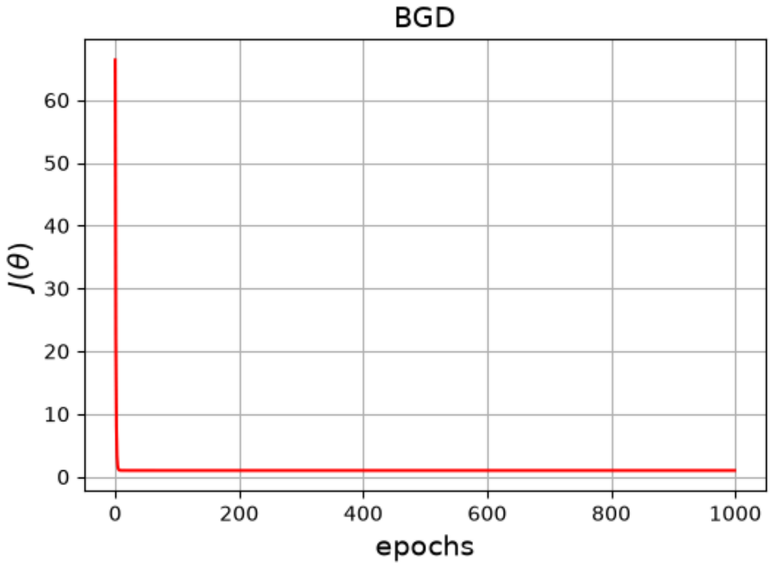


Figura 12. Costo de Batch Gradient Descent en 1,000 épocas.

Las tres variantes llegan prácticamente a la misma solución que la Ecuación Normal. La diferencia contra los valores reales (4 y 3) se debe al ruido que se agregó a los datos, no a los algoritmos.

PRUEBAS: FEATURE SCALING

Tabla 4. Verificación de la implementación propia.

Método	Resultado en las 3 columnas	Dif. máx. vs. Scikit-Learn
Normalización	mín = 0.0 y máx = 1.0	1.11×10^{-16}
Estandarización	μ = 0.0000 y σ = 1.0000	0.0

- Las dos funciones propias cumplen las propiedades esperadas en edad, salario y horas.
- Dan los mismos valores que MinMaxScaler y StandardScaler. La diferencia de 1.11×10^{-16} se debe al redondeo de punto flotante.

Tabla 5. Valores escalados de la edad.

Edad	Normalizada	Estandarizada
22	0.0000	-1.1872
55	0.8919	1.2016
49	0.7297	0.7673
39	0.4595	0.0434
38	0.4324	-0.0290
59	1.0000	1.4912
22	0.0000	-1.1872
51	0.7838	0.9121
27	0.1351	-0.8252
22	0.0000	-1.1872

BUGS CONOCIDOS

No hubieron bugs encontrados

CONCLUSIONES

- Las tres variantes de Gradient Descent llegaron prácticamente a la misma solución que la Ecuación Normal ($\theta_0 \approx 3.69$, $\theta_1 \approx 3.33$). La diferencia contra los valores reales (4 y 3) se debe al ruido de los datos, no a los algoritmos.
- SGD usó una sola muestra en cada una de sus 10,000 actualizaciones y quedó un poco más lejos de la solución exacta ($\theta_1 = 3.307$), lo que refleja el ruido de sus pasos. Mini-Batch, con grupos de 20, quedó más cerca ($\theta_1 = 3.334$).
- La normalización y la estandarización propias dieron los mismos valores que Scikit-Learn, con una diferencia máxima de 1.11×10^{-16}, que es solo redondeo. Esto confirma que las fórmulas quedaron bien implementadas.
- En el dataset, el salario estaba en decenas de miles y la edad y las horas en decenas. Después de escalar, las tres quedaron en rangos comparables, que es lo que necesita Gradient Descent para que ninguna variable domine el gradiente.

COMENTARIOS DE LOS INTEGRANTES

Esteban Martínez

Me enfoqué en la parte de Stochastic Gradient Descent. Lo que más me costó fue entender por qué el ruido de SGD puede ayudar a salir de mínimos locales. Al final me quedó claro que no siempre el camino más estable es el mejor.

Javier Osorio

Trabajé en Mini-Batch Gradient Descent y en la tabla comparativa. Comparar las tres variantes lado a lado me ayudó a ver que todas son el mismo algoritmo y que solo cambia cuántos datos se usan en cada paso.

Cesar Sosa

Me encargué del escalamiento de features y de los ejemplos con la edad. Calcular la normalización y la estandarización a mano me ayudó a entender qué hace cada fórmula, y no solo a usarla.

REFERENCIAS

Géron, A. (2025). *Hands-on machine learning with Scikit-Learn and PyTorch* (caps. 2 y 4). O'Reilly Media.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Raschka, S., Liu, Y. y Mirjalili, V. (2022). *Machine learning with PyTorch and Scikit-Learn* (caps. 2 y 4). Packt.

Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv*. https://doi.org/10.48550/arXiv.1609.04747

Scikit-learn developers. (s. f.). *Preprocessing data*. scikit-learn. https://scikit-learn.org/stable/modules/preprocessing.html

GRACIAS